

Celaya NFC 控制器 Android 软件开发

移植指南

文档版本 06
发布日期 2024-10-16

前言

概述

本文档从软件开发角度说明了将 NFC（Near Field Communication）控制器适配到 Android 系统的方法。主要包括软件总体框图、内核驱动移植方法、将 NFC 控制器适配到 AOSP 源码中的具体方法、配置文件、常见问题。

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修改记录

文档版本	发布日期	修改说明
06	2024-10-16	- 更新“2.2 获取驱动源码”中的文件描述。 - 更新“3.1.1 步骤一：获取 NFC 源码包”。 - 更新“3.1.2-步骤二：安装 NFC 等代码 Patch”的步骤 1。 - 更新“3.2.2 步骤二：安装 NFC 代码 Patch”的步骤 1。 - 更新“3.2.5 步骤五：NFCC 固件部署”。
05	2023-06-27	新增“2.5-测试驱动”小节。

文档版本	发布日期	修改说明
		新增“5.5-NFC discovery 命令参数错误”小节。
04	2023-02-06	- 更新“2.3 代码移植和适配”中的驱动目录和文件描述。 - 新增“2.3.4 安全函数库编译指导”与“2.3.5 DTS 配置”。 - 更新“3.1.3 步骤三：更新 NFC 参数配置文件”关于 libnfc-nci.conf 配置文件的说明。 - 更新“4 NFC 参数配置文件”中的 SET_RF_PARAM 配置参数说明。
03	2022-08-23	更新“3.1.4 步骤四：NFCC 固件部署”小节内容描述。
02	2022-06-20	- 更新“2.3 代码移植和适配”中表 2-1 的 phone(dir)说明。 - 更新“3.1.1 步骤一：获取 NFC 源码包”中获取第三方安全函数库源码的参考章节。 - 在“4 NFC 参数配置文件”的表 4-1 中更新 ISO_DEP_MAX_TRANSCEIVE、FORCE_FW_DNLD 参数的描述；新增 FORCE_DEV_PWROFF 参数的描述。 - 更新“5.1 驱动打开失败”小节。 - 更新“5.2 配置文件打开失败”中的日志提示信息。
01	2022-04-29	第一次正式版本发布。

目 录

前言 i

1 Android NFC 系统架构..... 1

2 NFC 内核驱动 3

2.1 驱动描述..... 3

2.2 获取驱动源码 3

2.3 代码移植和适配..... 4

2.3.1 oal_os.h OS 接口 5

2.3.2 oal_plat_op.h 文件操作和日志打印接口 6

2.3.3 board_q.c 板级接口 6

2.3.4 安全函数库编译指导 7

2.3.5 DTS 配置 9

2.4 编译内核..... 10

2.5 测试驱动..... 10

3 AOSP 协议栈 11

3.1 新版 Android SDK 11

3.1.1 步骤一：获取 NFC 源码包 11

3.1.2 步骤二：安装 NFC 等代码 Patch 11

3.1.3 步骤三：更新 NFC 参数配置文件 12

3.1.4 步骤四：NFCC 固件部署 12

3.1.5 步骤五：修改 Makefile 12

3.1.6 步骤六：安卓权限配置 13

3.1.7 步骤七：构建 NFC 和系统映像 13

3.1.8 步骤八：测试 NFC 功能..... 13

3.2 旧版 Android SDK	13
3.2.1 步骤一：获取 NFC 源码包	13
3.2.2 步骤二：安装 NFC 代码 Patch.....	14
3.2.3 步骤三：新增 NFC 参数配置文件	14
3.2.4 步骤四：更新 NFC 参数配置文件	14
3.2.5 步骤五：NFCC 固件部署	15
3.2.6 步骤六：修改 Makefile	15
3.2.7 步骤七：构建 NFC 和系统映像.....	15
3.2.8 步骤八：测试 NFC 功能.....	15
4 NFC 参数配置文件.....	16
5 常见问题.....	17
5.1 驱动打开失败	17
5.2 配置文件打开失败	17
5.3 HIDL 服务信息注册丢失.....	18
5.4 共享库打开失败.....	18
5.5 NFC discovery 命令参数错误.....	18
A 缩略语.....	19

插图目录

图 1-1 Android NFC 系统架构..... 1

表格目录

表 2-1 驱动代码说明 4

表 2-2 oal_os.h 接口函数说明 5

表 2-3 oal_plat_op.h 接口函数说明 6

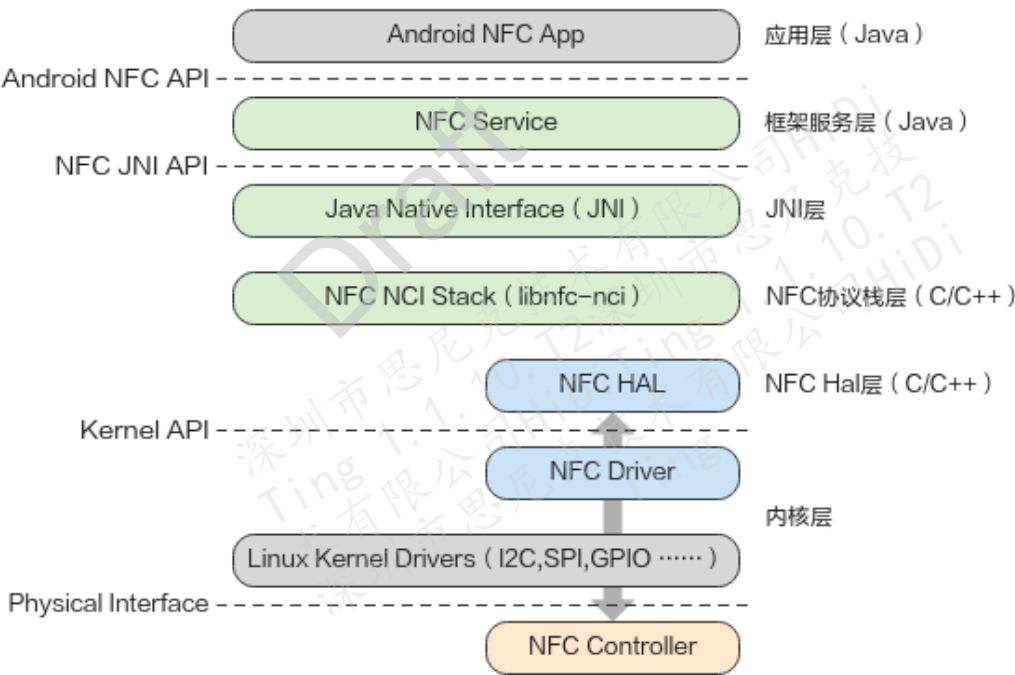
表 2-4 board_q.c 接口函数说明 7

表 4-1 libnfc-hal.conf 文件配置参数表 16

1 Android NFC 系统架构

Android NFC 系统架构如图 1-1 所示。

图1-1 Android NFC 系统架构



Android NFC 系统架构图说明如下：

- 框图颜色说明：灰色为用户自有模块，绿色为开源软件，蓝色为本方案软件，橙色为本方案硬件。
- NFC Driver 是 NFC 的内核驱动，用于访问底层 NFC 控制器资源，相关移植细节请参见 2 NFC 内核驱动章节。

- NFC HAL 是硬件抽象层，用于向上层提供硬件服务的抽象接口，是 [3 AOSP 协议栈](#) 章节 NFC 业务软件移植的主要部分。
- NFC NCI Stack 是本地的 NCI（NFC Controller Interface）协议栈，完成基于 NCI 的状态机控制。
- NFC JNI 是一个 Java/C++ 代码的中间转换层。
- NFC Service 作为框架服务层，对上层 App 提供接口服务。

2 NFC 内核驱动

- 2.1 驱动描述
- 2.2 获取驱动源码
- 2.3 代码移植和适配
- 2.4 编译内核
- 2.5 测试驱动

2.1 驱动描述

NFC 驱动通过 I2C 和底层的 NFC 控制器通讯，当驱动加载起来后，会提供给上层一个访问节点/dev/nfc_dev。集成商修改该驱动节点名称，同时需要修改协议栈配置文件“libnfc-hal.conf”中的 NFC_DEV_NODE 参数值（详细内容请参见 3 AOSP 协议栈章节）。

2.2 获取驱动源码

获取 NFC 驱动源码包，解压到 kernel 目录下的 drivers/nfc 路径下（原路径下部分文件会被覆盖，请提前备份）。解压后 Vendor NFC 驱动相关文件说明如下：

- Makefile：驱动代码的 makefile。
- Kconfig：驱动配置参数。
- comm_driver：驱动通用代码的子文件夹，具体文件请见源码包。
- adapter：具体产品下驱动源码的子文件夹，具体文件请见源码包。

获取依赖的第三方库为“libboundscheck”，获取链接为<https://gitee.com/openeuler/libboundscheck>。此库主要提供 memcpy、strcpy 等内存操作库函数的安全版本。NFC 驱动代码使用了安全版本的内存操作函数。

2.3 代码移植和适配

NFC 驱动需要通过简单的移植和适配，嵌入 kernel 环境才能编译运行。如 2.2 获取驱动源码小节介绍，驱动代码主要分为 comm_driver 模块和 adapter 模块，代码分别放在对应的目录。

- comm_driver 实现主要的驱动功能，不需要改动。
- adapter 需要用户适配板级接口，这些接口由 comm_driver 调用。adapter 中提供了 /dev/nfc_dev 文件节点，作为用户态的接口。

代码结构与详细描述如表 2-1 所示。

表2-1 驱动代码说明

目录	目录下的子文件夹和文件说明			
adapter(dir)	需要用户根据应用环境适配，实现固定的系统和板级接口。			
	oal(dir)	系统抽象层，适配不同 OS 接口。		
	-	linux(dir)	linux 系统，unix-like 已适配，用户不需要修改。	
		-	oal_os.h	os 接口适配。
			oal_plat_op.h	文件操作接口以及日志打印接口适配。
	product(dir)	产品板级接口适配。		
	-	phone(dir)	Android 和 Linux 环境适用	
		-	board_q.c	实现 board_nfc.h 定义的板级接口，这些接口被 comm_driver 模块调用，需要用户结合不同设备环境进行适配。

目录	目录下的子文件夹和文件说明			
			nfc_dev_q.h nfc_dev_q.c nfc_dev_att_r_q.c nfc_dev_att_r_q.h	/dev/nfc_dev 文件节点， 作为用户态的接口。
comm_driver(dir)	实现主要的驱动功能，不需要改动。			
	board_nfc.h	-		
	priv_nfc_driver.h	-		
	priv_nfc_driver.c	-		

2.3.1 oal_os.h OS 接口

需要适配实现接口，建议放在.h 文件中，通过 inline 或者函数宏实现，具体可以参考已有的代码实现。适配实现接口如表 2-2 所示。

表2-2 oal_os.h 接口函数说明

函数	功能说明
void oal_msleep(uint32_t time)	延时实现
void oal_udelay(uint32_t time)	
void oal_mutex_init(oal_mutex_t *lock)	互斥量实现
void oal_mutex_destroy(oal_mutex_t *lock)	
void oal_mutex_lock(oal_mutex_t *lock)	
oal_mutex_unlock(oal_mutex_t *lock)	
oal_spin_lock_init	临界段实现
oal_spin_lock_irqsave	
oal_spin_unlock_irqrestore	
void oal_init_waitqueue_head(oal_wait_queue_head_t *wq)	唤醒事件实现
int oal_wait_event_interruptible_timeout(oal_wait_queue_head_t	

函数	功能说明
wq, int condition, unsigned long timeout)	
void oal_wkup_event(oal_wait_queue_head_t wq)	

2.3.2 oal_plat_op.h 文件操作和日志打印接口

需要适配实现文件操作和日志打印接口，建议放在.h 文件中，通过 inline 或者函数宏实现，具体可以参考已有的代码实现。接口函数如表 2-3 所示。

表2-3 oal_plat_op.h 接口函数说明

函数	功能说明
oal_file_t *oal_file_open_readonly(const char *path)	只读打开文件。
oal_file_read_ext(oal_file_t *file, oal_loff_t pos, char *buf, uint32_t count)	读文件。
int32_t oal_file_close(oal_file_t *file)	关闭文件。
int oal_file_open_get_len(const char *path, oal_file_t **fp, unsigned int *file_len)	获取文件长度。
PS_PRINT_FUNCTION_NAME	打印当前函数名。
PS_PRINT_DBG(s, args...)	debug 级别打印。
PS_PRINT_INFO(s, args...)	info 级别打印。
PS_PRINT_SUC(s, args...)	打印执行成功。
PS_PRINT_WARNING(s, args...)	warning 级别打印。
PS_PRINT_ERR(s, args...)	error 级别打印。
PS_PRINT_ALERT(s, args...)	alert 级别打印。

2.3.3 board_q.c 板级接口

实现 AP 侧与 NFC 芯片之间的硬件接口操作，如 GPIO 控制、I2C 读写等。NFC 芯片的 I2C 地址是 0x28，时钟最高支持 full speed 400K。需要注意的是，需要在板级接口中实现 “dev_wakeup_host_gpio” 中断函数，并注册，中断函数中需要调用

“comm_driver” 模块中的 “nfc_wakeup_host_gpio_isr” 函数。具体可以参考已有的代码实现如表 2-4 所示。

表2-4 board_q.c 接口函数说明

函数	功能说明
int board_init(void *param)	板级初始化，主要是 GPIO，数据结构等等的初始化。
void board_deinit(void)	NFC 关闭时的去初始化动作。
void board_pwn_ctrl(nfc_enable_ctrl_t enable)	控制 NFC 芯片上下电。
void board_wkup_host_gpio_int_enable(nfc_enable_ctrl_t en)	dev_wakeup_host_gpio 中断使能控制。
int board_get_dev_wkup_host_status(void)	获取 dev_wakeup_host_gpio 状态。
int board_get_host_wkup_dev_status(void)	获取 host_wakeup_dev_gpio 状态。
void board_set_host_wkup_dev_value(int value)	设置 host_wakeup_dev_gpio 状态。
const char *board_get_fw_cfg_path(void)	获取固件文件存放路径。
int board_i2c_write(char *buf, int count)	通过 I2C 往 NFC 芯片写数据（默认地址 0x28）。
int board_i2c_read(char *buf, int count)	通过 I2C 从 NFC 芯片读数据（默认地址 0x28）。

2.3.4 安全函数库编译指导

- 1. 参照 2.2 获取驱动源码获取安全函数库源码。
- 2. 将如下文件添加到项目中，参加编译。

源文件	文件说明
-----	------

源文件	文件说明
include/securec.h	安全函数对外接口头文件。
include/securectype.h	对外接口辅助头文件，被 securec.h 包含。
src/memcpy_s.c	提供 memcpy_s 函数。
src/memmove_s.c	提供 memmove_s 函数。
src/memset_s.c	提供 memset_s 函数。
src/strcpy_s.c	提供 strcpy_s 函数。
src/strncpy_s.c	提供 strncpy_s 函数。
src/strcat_s.c	提供 strcat_s 函数。
src/strncat_s.c	提供 strncat_s 函数。
src/secureutil.c	提供获取安全函数版本号的函数。
src/snprintf_s.c	提供 snprintf_s 和 snprintf_truncated_s 函数。
src/vsnprintf_s.c	提供 vsnprintf_s 和 vsnprintf_truncated_s 函数。
src/sprintf_s.c	提供 sprintf_s 函数。
src/vsprintf_s.c	提供 vsprintf_s 函数。
src/secureprintoutput_a.c(包含 output.inl)	该文件为格式化输出公共函数，不提供浮点数以及宽字符处理功能。
src/sscanf_s.c	提供 sscanf_s 函数。
src/vsscanf_s.c	提供 vsscanf_s 函数。
src/secureinput_a.c(包含 input.inl)	该文件为格式化输入公共函数，不提供浮点数以及宽字符处理功能。
src/strtok_s.c	提供 strtok_s 函数。

3. 修改 Makefile 适配内核编译。

```

MK_FILE_PATH := $(abspath $(lastword $(MAKEFILE_LIST)))
PRIV_KSECUREC_BUILTIN_DIR := $(dir $(MK_FILE_PATH))
PRIV_KSECUREC_INCLUDES := -I$(PRIV_KSECUREC_BUILTIN_DIR)include

```

```

PRIV_KSECUREC_SRC := src

EXTRA_CFLAGS += $(PRIV_KSECUREC_INCLUDES)

ksecurec-objs := memcpy_s.o memmove_s.o memset_s.o  secureutil.o  strcat_s.o
strcpy_s.o strncat_s.o strncpy_s.o sprintf_s.o vsprintf_s.o snprintf_s.o vsnprintf_s.o
secureprintoutput_a.o sscanf_s.o vsscanf_s.o secureinput_a.o
ksecurec-objs := $(addprefix $(PRIV_KSECUREC_SRC)/,$(ksecurec-objs))
obj-y += $(ksecurec-objs)

```

2.3.5 DTS 配置

NFC 驱动需要用到板级 I2C 接口及 GPIO 接口， 需要根据板级硬件连接， 在 DTS 中适对应应的资源， 配置 I2C 速率 full speed 400K， 时钟占空比 40%~60%， 建议 50%。 示例参考如下。 请注意， 示例只是参考， 取决于主芯片平台， 可配置的参数或者格式不一样。

```

&i2c6 {                                                                 /* 请根据板级
分配的I2C通道号来修改 */

    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&i2c6_pins_a>;
    pinctrl-1 = <&i2c6_pins_sleep_a>;
    i2c-scl-rising-time-ns = <185>;
    i2c-scl-falling-time-ns = <20>;
    status = "okay";

    nfc@28{                                                                 /* 请根据板级分
配的I2C地址来修改 */

        compatible = "nfc_module,nfc";
        status = "ok";
        reg = <0x28>;

        nfc.gpio_wakeup_host = <&gpio1 7 0>;          /* 请根据板级wakeup host GPIO连接情况
修改 */

        nfc.gpio_poweron = <&gpio1 4 0>;                /* 请根据板级PWREN GPIO连接情况修
改 */

        nfc.gpio_wakeup_device = <&gpio1 6 0>;        /* 请根据板级wakeup device GPIO连接情况
修改 */

    };
};

```

2.4 编译内核

在 menuConfig 阶段，选择 Vendor NFC Driver 参与内核编译。

```
Device Drivers ---->
<*> Vendor NFC Driver
```

完成编译后，驱动就在内核映像文件中了。

2.5 测试驱动

在 sys/nfc 目录下存在 NFC 相关的节点，执行以下两条命令可对 nfc_download 进行写入字符 1，然后读取检查 nfc_get_result 是否为 1，从而判断驱动加载是否成功。

```
echo "1" > /sys/nfc/nfc_download      // 芯片上电，加载固件
cat /sys/nfc/nfc_get_result           // 检查驱动加载是否成功，1为成功
```

3 AOSP 协议栈

本章节将分步骤讲解如何利用提供的 NFC 源码包移植适配到 AOSP 11_0_0_r3 版本。移植到 Android 其他版本需要部分 API 的适配。

3.1 新版 Android SDK

3.2 旧版 Android SDK

3.1 新版 Android SDK

本小节适用于较新版本的 Android SDK 软件适配，此处以 Android 11 为例进行介绍。

3.1.1 步骤一：获取 NFC 源码包

获取 NFC 源码包的方法如下：

- 获取 NFC 源码包，将源码包中 driver 和 nfc 文件夹解压到 android 11 根目录（用 \$(AOSP_root) 表示）下的 “\$(AOSP_root)/vendor/vxx” 路径下。
- 自行下载获取 “2.2 获取驱动源码” 小节介绍的第三方安全函数库源码。

3.1.2 步骤二：安装 NFC 等代码 Patch

安装 NFC 等代码 Patch 步骤如下：

步骤 1 进入 android 源码根目录 “\$(AOSP_root)/”，执行脚本

“./\$(AOSP_root)/vendor/vxx/nfc/v11/install_NFC_AOSP11.sh \$(AOSP_root)” 进行代码 Patch，脚本执行依赖输入变量为 android 协议栈源码根目录，如果执行报错请检查输入的路径是否正确。

步骤 2 以上脚本执行完毕后，会给下面相关目录的一些代码打 patch，具体内容参见 install_NFC_AOSP11.sh 脚本。

步骤 3 把下载的第三方安全函数库源码包内的子文件夹全拷贝到 “\$(AOSP_root)/hardware/vxx/nfc/secure_c” 目录下。

----结束

3.1.3 步骤三：更新 NFC 参数配置文件

根据 5 常见问题内容以及最终产品的实际情况，把源码包

“vendor/vxx/nfc/v11/hal/conf” 下 hal 层配置文件的相关配置参数进行适当的修改，最终会集成到产品设备 Android 系统的目录下（路径参见 device-nfc.mk）：

- libnfc-hal.conf：包含 HAL 层以及 Vendor 底层私有配置参数。
- libnfc-nci.conf：类似该名的文件，具体可参考上层 Host 协议栈 NfcConfig::loadConfig()所需加载的配置文件，该文件一般可在安卓开源代码 /device 目录下找到。

3.1.4 步骤四：NFCC 固件部署

源码包里有 NFCC (NFC Controller) 的固件，可放置到源码包内 “device-nfc.mk” 里描述的 AOSP 源码路径下，构建系统映像的时候会拷贝到对应的设备路径下。具体配置方法客户可参考 “device-nfc.mk” 自己修改，部分脚本配置内容如下：

- PRODUCT_COPY_FILES += \
- vendor/\$(VENDOR_DIR)/nfc/firmware/RAM.bin:\$(FW_PATH)/RAM.bin
- vendor/\$(VENDOR_DIR)/nfc/firmware/nfc_cfg:\$(FW_PATH)/nfc_cfg

请注意，这两个文件需要提前保存在 host 端，占用存储空间一般不超过 150KB，具体视用户所得固件文件大小情况而定。另外，默认拷贝路径可以通过搜索和修改平台驱动代码里的默认路径、上述 mk 文件配置路径、以及 nfc_cfg 里的默认路径达成。

3.1.5 步骤五：修改 Makefile

修改 Makefile 的方法如下：

- 可参考 NFC 源码包内 “device-nfc.mk” 的内容做适当修改，在 device.mk 文件中（一般路径/device/xxx/device.mk），增加 “\$(call inherit-product, vendor/vxx/nfc/v11/device-nfc.mk)” 语句。

- 可参考 NFC 源码包内 “BoardConfig-nfc.mk” 的内容做适当修改，在 BoardConfig.mk 文件中（一般路径/device/xxx/BoardConfig.mk），增加 “-include vendor/vxx/nfc/v11/BoardConfig-nfc.mk” 语句。

3.1.6 步骤六：安卓权限配置

- NFC 访问节点权限和文件路径权限。

可参考安装包内 vendor/vxx/nfc/v11/init.nfc.rc 配置文件，主要配置项如下：

- mkdir /data/vendor 0777 nfc nfc
- mkdir /data/vendor/nfc 0777 nfc nfc
- chmod 0660 /dev/nfc_dev
- chown nfc nfc /dev/nfc_dev

- 安全策略配置文件。

可参考安装包内 “vendor/vxx/nfc/v11/sepolicy” 文件夹下的配置文件内容，具体内容此处不列举。提供的是参考文件，需要在产品设备开机后运行宽容模式，检查拒绝事件记录并做相应的策略配置的修改。

3.1.7 步骤七：构建 NFC 和系统映像

上面配置完成后，即可构建 NFC 和整个系统映像。请确保 3 AOSP 协议栈提到的 Vendor NFC 驱动已经被集成到内核 boot 映像文件中。把所有映像文件烧录到新设备中，验证可以正常开机。

3.1.8 步骤八：测试 NFC 功能

在 Android 的 Settings 设置页面，选择 “更多连接”，可以看到 NFC 开关。打开 NFC 开关，正常点亮后，即可测试 NFC 业务。

3.2 旧版 Android SDK

本小节适用于较老版本的 Android SDK 软件适配，此处以 Android 4.4 为例进行介绍。

3.2.1 步骤一：获取 NFC 源码包

获取 NFC 源码包的方法如下：

- 获取 NFC 源码包，将源码包中 driver 和 nfc 文件夹解压到 android 4.4 根目录（用\$(AOSP_root)表示）下的 “\$(AOSP_root)/vendor/vxx” 路径下。
- 自行下载获取 “2.2 获取驱动源码” 小节介绍的第三方安全函数库源码，将安全函数库的源文件和头文件分别解压到 “\$(AOSP_root)/external/libnfc-nci/halimpl/bcm2079x/secure_src” 和 “\$(AOSP_root)/external/libnfc-nci/halimpl/bcm2079x/secure_include” 路径下。

3.2.2 步骤二：安装 NFC 代码 Patch

安装 NFC 等代码 Patch 步骤如下：

- 步骤 1 进入 Android 4.4 源码根目录 “\$(AOSP_root)”，在根目录执行解压出的脚本 “\$(AOSP_root)/vendor/vxx/nfc/v4.4/install_NFC_AOSP4.4.sh \$(AOSP_root)” 进行代码 Patch，脚本执行依赖输入参数为 android 协议栈源码根目录，如果执行报错请检查输入的路径是否正确。
- 步骤 2 以上脚本执行完毕后，会给下面相关目录的一些代码打 patch，并将驱动代码拷贝至相应路径下，具体内容参见 install_NFC_AOSP4.4.sh 脚本。

----结束

3.2.3 步骤三：新增 NFC 参数配置文件

在 /etc/ 路径下添加配置文件 libnfc-brcm.conf

配置文件中添加内容：

SCREEN_OFF_POWER_STATE=0x01（防止低功耗 sleep 关闭 hal 层）

如果用户只启用卡模拟模式，建议配置如下变量：

POLLING_TECH_MASK=0x00（关闭读卡器模式）

P2P_LISTEN_TECH_MASK=0x00（关闭 p2p listen 模式）

3.2.4 步骤四：更新 NFC 参数配置文件

根据 5 常见问题内容以及最终产品的实际情况，把源码包

“\$(AOSP_root)/vendor/vxx/nfc/v4.4” 下 hal 层配置文件的相关配置参数进行适当的修改，最终会集成到产品设备 Android 系统的目录下（路径参见 device-nfc.mk）：

- libnfc-hal.conf：包含 HAL 层以及 Vendor 底层私有配置参数。

3.2.5 步骤五：NFCC 固件部署

源码包里有 NFCC (NFC Controller) 的固件，可放置到源码包内 “device-nfc.mk” 里描述的 AOSP 源码路径下，构建系统映像的时候会拷贝到对应的设备路径下。具体配置方法客户可参考 “device-nfc.mk” 自己修改，部分脚本配置内容如下：

- `PRODUCT_COPY_FILES += \`
- `vendor/$(VENDOR_DIR)/nfc/firmware/RAM.bin:$(FW_PATH)/RAM.bin`
- `vendor/$(VENDOR_DIR)/nfc/firmware/nfc_cfg:$(FW_PATH)/nfc_cfg`

请注意，这两个文件需要提前保存在 host 端，占用存储空间一般不超过 150KB，具体视用户所得固件文件大小情况而定。另外，默认拷贝路径可以通过搜索和修改平台驱动代码里的默认路径、上述 mk 文件配置路径、以及 nfc_cfg 里的默认路径达成。

3.2.6 步骤六：修改 Makefile

- 可参考 NFC 源码包内 “device-nfc.mk” 的内容做适当修改，在 device.mk 文件中（一般路径/device/xxx/device.mk），增加 “\$(call inherit-product, \$(AOSP_root)/vendor/vxx/nfc/v4.4/device-nfc.mk)” 语句。

3.2.7 步骤七：构建 NFC 和系统映像

上面配置完成后，即可构建 NFC 和整个系统映像。请确保 3 AOSP 协议栈提到的 Vendor NFC 驱动已经被集成到内核 boot 映像文件中。把所有映像文件烧录到新设备中，验证可以正常开机。

3.2.8 步骤八：测试 NFC 功能

在 Android 的 Settings 设置页面，选择 “更多连接”，可以看到 NFC 开关。打开 NFC 开关，正常点亮后，即可测试 NFC 业务。

4 NFC 参数配置文件

3.1.3 步骤三：更新 NFC 参数配置文件小节提到的配置文件，可在 NFC 初始化的时候配置 HAL 层的相关参数。在将 NFC 集成到新的设备中时，需要注意相关的配置参数是否需要修改。本文仅列举部分示例参数说明，详细内容和参数注释请参见 NFC 安装包内的示例配置文件。libnfc-hal.conf 文件配置参数如表 4-1 所示。

表4-1 libnfc-hal.conf 文件配置参数表

参数	描述
LOG_HAL_LOGLEVEL	HAL 模块的日志等级。
LOG_NCIX_LOGLEVEL	NCI 发送的日志等级。
LOG_NCIR_LOGLEVEL	NCI 接收的日志等级。
LOG_TML_LOGLEVEL	传输映射层模块的日志等级。
NFC_DEV_NODE	设备节点名字。
PRESENCE_CHECK_ALGORITHM	Iso-Dep 协议卡的在位检测算法。
NFA_PROPRIETARY_CFG	支持的私有协议配置。
ISO_DEP_MAX_TRANSCEIVE	Iso-Dep 支持 APDU 的最大长度。
FORCE_FW_DNLD	开 NFC 的时候强制做上下电和加载 FW。
FORCE_DEV_PWROFF	关 NFC 的时候强制下电。
SET_RF_PARAM	RF 定制化参数配置的载荷内容。

5 常见问题

5.1 驱动打开失败

5.2 配置文件打开失败

5.3 HIDL 服务信息注册丢失

5.4 共享库打开失败

5.5 NFC discovery 命令参数错误

5.1 驱动打开失败

在驱动加载有问题或者设备节点权限有问题时，日志中出现如下信息：

```
Open dev node failed!
```

此时可查看设备节点（默认是/dev/nfc_dev）的权限信息，确保 NFC 具有读写权限。
如果权限没问题时，可查看内核日志，检查驱动加载过程中的具体错误信息。

5.2 配置文件打开失败

在配置文件打开失败时，日志中出现如下信息：

```
Cfg file parse failed: XXXX
```

此时需要查看相关的配置文件是否放置在准确的路径。

5.3 HIDL 服务信息注册丢失

某些 hidl 接口服务未在 framework 或者 manifest 文件中申明时，日志中出现如下信息：

```
Hw servicemanager: getTransport: Cannot find entry XXXXXX in either framework or device manifest
```

通常可以在设备的 manifest.xml 文件中申明相关的 hidl 接口服务。

5.4 共享库打开失败

某些共享库打开失败时，日志中出现如下信息：

```
Dlopen failed: library XXXX not found
```

检查相应的共享库是否放置在准确的路径、编译共享库的流程和结果是否正确可用等。

5.5 NFC discovery 命令参数错误

设备打开 nfc 功能初始化后，发现“2103”开头的 discovery 命令反复下发很多次，并且存在非法参数“FF”如下所示：

```
VxxNfcNciX: NciX > 21030D0600010101020180010601FF01
```

修改 libnfc-hal.conf 的配置选项：NFA_PROPRIETARY_CFG，把倒数第三个字节改成 70，倒数第四个改成 80，倒数第五个改成 81。重新拷贝到设备上，查看问题是否解决。

A 缩略语

H

HAL	Hardware Abstract Layer	硬件抽象层
------------	-------------------------	-------

N

NCI	NFC Controller Interface	NFC 控制器接口
NFC	Near Field Communication	近场通信
NFCC	NFC Controller	NFC 控制器